1. Implementation Status

Part 1(a) - GPT-2 Forward Pass: I completed this part successfully, implementing the full GPT-2 forward pass including token and position embeddings, all 12 transformer blocks with multi-head causal self-attention, final layer norm, and the language modeling head with weight tying.

Part 1(b) - Text Generation with KV Caching: I completed this part successfully, implementing autoregressive token generation with KV caching, temperature-controlled sampling, and top-p nucleus sampling.

Part 1(c) - Pre-Training on OpenWebText: I completed this part successfully, implementing a pre-training loop that trains the GPT-2 language model on the provided OpenWebText subset using a standard next-token prediction objective.

Part 2(a) - Adapting GPT-2 for Text Classification: I completed this part successfully, implementing the GPT2ForSequenceClassification class that reuses the pre-trained GPT-2 with a linear classification head on top, using last-token pooling to produce logits over 20 classes.

Part 2(b) - Training Loop for Fine-Tuning: I completed this part successfully, implementing a full training loop that fine-tunes the classifier on the 20 Newsgroups dataset and achieves above 65% validation accuracy.

All tests in hw2_test.py pass.

```
========================== test session starts ==========================
platform linux -- Python 3.10.12, pytest-9.0.1, pluggy-1.5.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /partition2/Partition2/hw2
plugins: repeat-0.9.3, anyio-4.6.2.post1, xdist-3.6.1, deepeval-2.5.1, langsmith-0.3.7
collected 6 items

tests/hw2_test.py::test_loads_and_forward_pass PASSED                [ 16%]
tests/hw2_test.py::test_probability_tolerance PASSED                 [ 33%]
tests/hw2_test.py::test_greedy_sampling PASSED                       [ 50%]
tests/hw2_test.py::test_nucleus_sampling PASSED                      [ 66%]
tests/hw2_test.py::test_classifier_loads_and_forward_pass PASSED     [ 83%]
tests/hw2_test.py::test_classifier_accuracy PASSED                   [100%]Running teardown with pytest sessionfinish...


====================== 6 passed in 3212.56s (0:53:32) ======================
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1773939030.377069 2504285 init.cc:232] grpc_wait_for_shutdown_with_timeout() timed out.
davidx@davidx-XPS-15:~/Downloads/Partition2/hw2$
```

```
davidx@davidx-XPS-15:~/Downloads/Partition2/hw2/src$ python3 train.py
Epoch 0, Batch 0, Loss: 14.1031
Epoch 0, Batch 100, Loss: 2.6585
Epoch 0, Batch 200, Loss: 2.6929
Epoch 0, Batch 300, Loss: 3.0795
Epoch 0, Batch 400, Loss: 1.6542
Epoch 0, Batch 500, Loss: 1.0873
Epoch 0, Batch 600, Loss: 1.0860
```

```
Epoch 0, Batch 700, Loss: 0.8283
Epoch 0, Batch 800, Loss: 1.7104
Epoch 0, Batch 900, Loss: 1.1602
Epoch 0, Batch 1000, Loss: 0.1331
Epoch 0, Batch 1100, Loss: 1.1863
Epoch 0, Batch 1200, Loss: 0.9425
Epoch 0, Batch 1300, Loss: 1.5983
Epoch 0, Batch 1400, Loss: 1.6891
Epoch 0, Batch 1500, Loss: 1.2394
Epoch 0, Batch 1600, Loss: 1.3402
Epoch 0, Batch 1700, Loss: 1.5136
Epoch 0, Batch 1800, Loss: 1.2617
Epoch 0, Batch 1900, Loss: 0.5916
Epoch 0, Batch 2000, Loss: 1.2802
Epoch 0, Batch 2100, Loss: 0.0741
Epoch 0, Batch 2200, Loss: 0.7678
Epoch 0, Batch 2300, Loss: 0.9112
Epoch 0, Batch 2400, Loss: 0.7353
Epoch 0, Batch 2500, Loss: 0.6216
Epoch 0, Batch 2600, Loss: 0.9154
Epoch 0, Batch 2700, Loss: 1.5426
Epoch 0 average training loss: 1.4614
Epoch 0 validation accuracy: 0.6759 (67.59%)
Epoch 1, Batch 0, Loss: 1.5531
Epoch 1, Batch 100, Loss: 0.9334
Epoch 1, Batch 200, Loss: 0.6713
Epoch 1, Batch 300, Loss: 0.4167
Epoch 1, Batch 400, Loss: 0.0594
Epoch 1, Batch 500, Loss: 0.2781
Epoch 1, Batch 600, Loss: 0.8664
Epoch 1, Batch 700, Loss: 0.0184
Epoch 1, Batch 800, Loss: 0.3863
Epoch 1, Batch 900, Loss: 0.0690
Epoch 1, Batch 1000, Loss: 1.0722
Epoch 1, Batch 1100, Loss: 1.7289
Epoch 1, Batch 1200, Loss: 0.5394
Epoch 1, Batch 1300, Loss: 0.6236
Epoch 1, Batch 1400, Loss: 1.0741
Epoch 1, Batch 1500, Loss: 0.8601
Epoch 1, Batch 1600, Loss: 0.3401
Epoch 1, Batch 1700, Loss: 0.0707
Epoch 1, Batch 1800, Loss: 0.0598
Epoch 1, Batch 1900, Loss: 0.1687
Epoch 1, Batch 2000, Loss: 1.6057
Epoch 1, Batch 2100, Loss: 0.7112
```

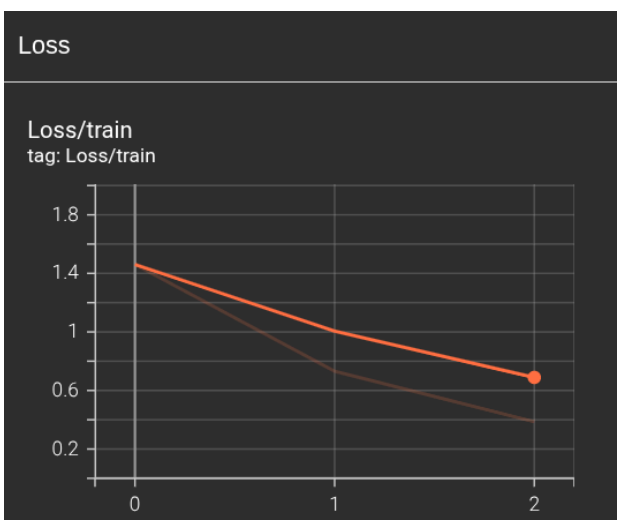
```
Epoch 1, Batch 2200, Loss: 2.4143
Epoch 1, Batch 2300, Loss: 0.6833
Epoch 1, Batch 2400, Loss: 1.0473
Epoch 1, Batch 2500, Loss: 0.0017
Epoch 1, Batch 2600, Loss: 1.0120
Epoch 1, Batch 2700, Loss: 0.5201
Epoch 1 average training loss: 0.7309
Epoch 1 validation accuracy: 0.7145 (71.45%)
Epoch 2, Batch 0, Loss: 0.5451
Epoch 2, Batch 100, Loss: 1.0581
Epoch 2, Batch 200, Loss: 0.0002
Epoch 2, Batch 300, Loss: 3.0742
Epoch 2, Batch 400, Loss: 0.0354
Epoch 2, Batch 500, Loss: 0.0001
Epoch 2, Batch 600, Loss: 0.8832
Epoch 2, Batch 700, Loss: 0.0271
Epoch 2, Batch 800, Loss: 0.0001
Epoch 2, Batch 900, Loss: 0.0006
Epoch 2, Batch 1000, Loss: 0.5271
Epoch 2, Batch 1100, Loss: 0.8372
Epoch 2, Batch 1200, Loss: 0.0003
Epoch 2, Batch 1300, Loss: 0.0023
Epoch 2, Batch 1400, Loss: 0.0216
Epoch 2, Batch 1500, Loss: 0.0008
Epoch 2, Batch 1600, Loss: 1.3600
Epoch 2, Batch 1700, Loss: 0.8653
Epoch 2, Batch 1800, Loss: 0.3441
Epoch 2, Batch 1900, Loss: 0.0004
Epoch 2, Batch 2000, Loss: 1.1607
Epoch 2, Batch 2100, Loss: 0.7996
Epoch 2, Batch 2200, Loss: 0.2371
Epoch 2, Batch 2300, Loss: 0.0003
Epoch 2, Batch 2400, Loss: 2.5138
Epoch 2, Batch 2500, Loss: 0.0036
Epoch 2, Batch 2600, Loss: 0.6384
Epoch 2, Batch 2700, Loss: 0.0001
Epoch 2 average training loss: 0.3861
Epoch 2 validation accuracy: 0.6975 (69.75%)
Saved classifier checkpoint to ../checkpoints/classifier_model.pth
```

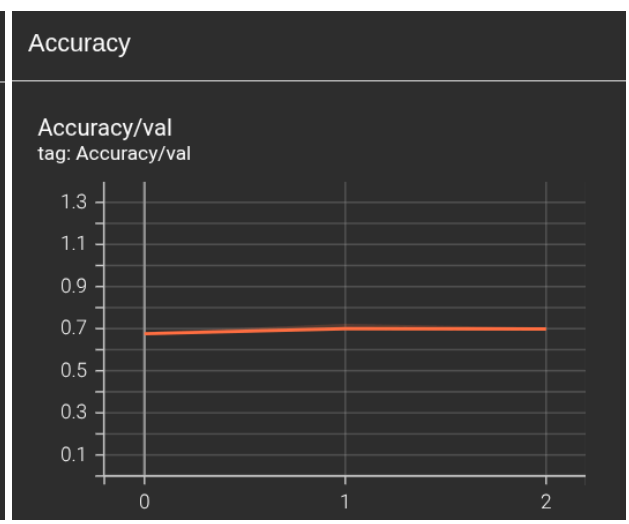
2. Training Techniques, Loss, and Validation Accuracy Curves

The fine-tuning pipeline uses transfer learning by initializing the classifier's transformer backbone from the pre-trained GPT-2 language model checkpoint, which gives the model a strong foundation of language understanding before it sees any labeled classification data. I chose AdamW as the optimizer with a learning rate of 5e-5 and weight decay of 0.01, which is the industry-standard configuration for fine-tuning pre-trained transformers; AdamW decouples weight decay from the adaptive learning rate updates, leading to better generalization compared to vanilla Adam. The learning rate of 5e-5 is deliberately small to avoid catastrophic forgetting, where aggressively updating the pre-trained weights would destroy the useful representations learned during language model pre-training. Cross-entropy loss was used as the training objective, which is the standard loss function for multi-class classification since it directly optimizes the log-likelihood of the correct class and pairs naturally with the softmax output distribution. Gradient clipping with a max norm of 1.0 was applied to prevent exploding gradients, which can occur early in fine-tuning when the randomly initialized classification head produces large loss values that propagate unstable gradients through the deep network. The training data is shuffled at each epoch to prevent the model from learning spurious ordering patterns, while the input sequences are truncated and padded to a uniform length of 512 tokens to balance capturing sufficient document context with manageable GPU memory usage. Training was run for 3 epochs, which is typical for fine-tuning pre-trained models on moderately sized datasets since the backbone already has strong representations and only needs light adaptation.

The model achieved a **final accuracy of 0.7145** after 6 hours and 35 minutes of training.

TensorBoard Results





3. Challenges and Solutions

My biggest challenge was implementing KV caching for autoregressive generation. Specifically, I struggled with figuring out how to properly share and persist the cache of previously computed key and value vectors across generation steps so that each new token could attend to the full history without redundantly recomputing attention over all prior tokens. Getting the in-place update mechanism right, where the forward pass writes updated key-value pairs back into the same list structure that gets passed into the next step. This required careful reasoning about Python reference semantics and how list mutation works versus reassignment.

I also had significant difficulty keeping track of the various tensor dimensions throughout the attention mechanism. The model dimension of 768 gets projected to 2304 (which is 3 times 768, for the fused Q/K/V projection), then split back into three 768-dimensional tensors, each of which is further reshaped into 12 heads of 64 dimensions each. The MLP intermediate dimension of 3072 (4 times 768) added another size to track. Keeping all these dimensions straight, especially with the transposes needed to go from [B, T, n_head, d_head] to [B, n_head, T, d_head] for the attention computation and back again -- was a persistent source of shape mismatch errors that required careful debugging with print statements and reference back to the GPT-2 architecture diagram.

4. AI Usage and Collaboration

I used AI tools extensively throughout this assignment, primarily with Claude Opus, to assist with planning, learning, and debugging. Before writing any code, I used AI to outline the implementation steps for each part of the assignment, breaking down the GPT-2 architecture into manageable components and establishing the order in which to implement and test them. This planning phase was critical for understanding the dependencies between components. For example, knowing that the forward pass needed to be working before KV caching could be tested, and that the language model needed to be complete before the classifier could reuse its backbone.

I relied heavily on AI to read up on PyTorch documentation and understand what methods and parameters various objects provide, such as the differences between nn.Linear and custom transposed linear layers, how nn.Embedding works, and the exact behavior of functions like torch.tril, F.softmax, and torch.multinomial. AI was also invaluable for understanding design choices that are considered standard practice in the field, such as why AdamW is preferred over Adam for fine-tuning, why cross-entropy loss is the right choice for multi-class classification, and the intuition behind gradient clipping and weight decay. A significant portion of my AI usage was dedicated to debugging. I frequently pasted error messages and stack traces from running the pytest suite and used AI to help decipher what the errors meant, trace them back to the specific line or dimension mismatch causing the issue, and suggest targeted fixes.